

Course Code	: MCS-207
Course Title	: Database Management Systems
Assignment Number	: BCA_NEW(III)-207/Assignment/2025-26
Maximum Marks	: 100
Weightage	: 30%
Last Date for Submission	: 31th October 2025 (for July Session) 30th April, 2026 (for January session)

There are four questions in this assignment, which carries 80 marks. Rest 20 marks are for viva voce. You may use illustrations and diagrams to enhance the explanations. Please go through the guidelines regarding assignments given in the Programme Guide for the format of the presentation. The answer to each part of the question should be confined to about 300 words. Make suitable assumption, if any.

Question 1: (Covers Block 1)

(4+4+4+4+4=20 Marks)

(a) What are the limitations of file-based systems? How does a database approach resolve those limitations? Also, explain the physical architecture of a database management system (DBMS).

Solution: Limitations of File-Based Systems

- **Data Redundancy:** File-based systems store the same data in multiple files, leading to unnecessary duplication and wastage of storage space.
- **Data Inconsistency:** Because duplicated data might not be updated everywhere, inconsistencies can occur and compromise data integrity.
- **Data Isolation:** Data is scattered across separate files, making it difficult to access related data for queries and reporting.
- **Data Dependence:** Changes in the file structure require application programs to be rewritten, making the system inflexible.
- **Poor Security and Concurrency:** Limited mechanisms to control data access and handle multiple users, which can lead to unauthorized access and data conflicts.

How Database Approach Resolves Limitations

- **Minimizes Redundancy:** Data is centrally stored, eliminating unnecessary duplication and improving consistency.
- **Data Integrity:** Constraints and rules ensure accuracy and consistency of data across all applications and users.
- **Data Independence:** Structural changes can be made to the database without affecting application programs.

- **Enhanced Security and Concurrency:** Robust security features and transaction management prevent unauthorized access and support multiple users.

Physical Architecture of DBMS

- **Internal/Physical Level:** Lowest tier handling actual storage of data on devices. Includes data files, indexes, and data dictionary storing metadata. Manages how data is physically stored and retrieved.
- **Components:** DBMS engine (manages storage and access), storage manager (controls storage allocation), buffer manager (handles memory/cache), and metadata catalog (contains schema information).

Thus, DBMS resolves the inefficiencies of file-based systems and has a layered architecture to manage data storage efficiently.

(b) Explain the following terms in the context of a relational model with the help of one example of each:

(i) Key constraints

Solution: A key constraint ensures that a particular attribute or set of attributes uniquely identifies each tuple (row) in a relation (table).

Example: In a Student table, the **StudentID** must be unique for every student.

(ii) Domain constraints

Solution: Domain constraints specify the permissible values for an attribute. The values must be from a defined domain (like data type or specific range).

Example: An Age attribute can only have integer values between 1 and 120.

(iii) Candidate key

Solution: A candidate key is a minimal set of attributes that can uniquely identify a tuple. There can be multiple candidate keys, but one is selected as the primary key.

Example: In an Employee table, both **EmployeeID** and **Email** can be candidate keys.

(iv) Select operation

Solution: Select operation retrieves rows from a relation that satisfy a specified condition.

Example: $\sigma(\text{Age} > 25)(\text{Employee})$ selects employees older than 25 years.

(v) Project Operation

Solution: Project operation retrieves specific columns (attributes) from a relation.

Example: $\pi(\text{Name, Age})(\text{Student})$ retrieves only Name and Age columns from the Student table.

(vi) Equijoin Operation

Solution: Equijoin combines tuples from two relations based on equality of attribute values.

Example: Joining Employee and Department tables where Employee.DeptID = Department.DeptID.

(vii) Set Difference operation

Solution: Set difference returns tuples in one relation but not in another.

Example: Students who have not registered for a course = All_Students - Registered_Students.

(viii) Referential Integrity constraint

Solution: Ensures foreign key values in one relation match primary key values in the related relation or are null.

Example: An Orders table has CustomerID as foreign key that must exist in the Customers table.

(c) A departmental store maintains the inventory of all the items that it sells using a database system. In addition, this system is used for keeping a record of the sales of items to its registered customers.

This database is used to find the following details by the staff of the departmental store:

- List of the items that are out of stock.
- List of customers and the items purchased by them.
- List of all the items in the store.

Draw an ER diagram for the departmental store. Specify key attributes and constraints on each entity type and on each relationship type. Note any unspecified requirement and make appropriate assumptions to make the specification complete.

Solution: ER Diagram for Departmental Store Inventory and Sales System

Entities and Key Attributes:

1. Item

- Key Attribute: *ItemID*
- Other Attributes: ItemName, Price, QuantityInStock

2. Customer

- Key Attribute: *CustomerID*
- Other Attributes: *CustomerName*, *ContactInfo*

3. Sale (Relationship entity, representing purchase)

- Attributes: *SaleID* (key), *SaleDate*, *QuantityPurchased*

Relationships:

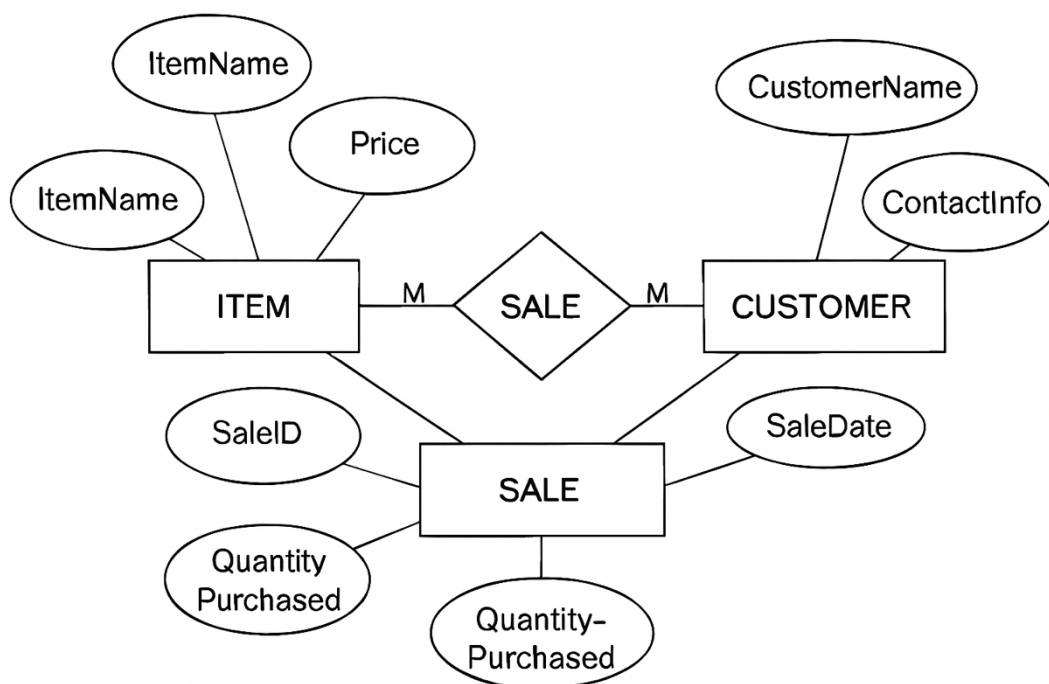
- **Purchases** (between Customer and Item)

- Type: Many-to-Many (A customer can buy many items; an item can be purchased by many customers)
- Attributes: *QuantityPurchased* (in Sale entity)
- Constraints:
 - *CustomerID* in Sale references Customer entity (foreign key).
 - *ItemID* in Sale references Item entity (foreign key).

Assumptions:

- Each item has a unique identifier *ItemID*.
- Customers must be registered to be recorded as buyers.
- *QuantityInStock* helps find out if the item is out of stock (*QuantityInStock* = 0).
- Sales are recorded with date and quantity for each purchase.

ER Diagram



ER Diagram Description:

- **Item** (ItemID, ItemName, Price, QuantityInStock)
- **Customer** (CustomerID, CustomerName, ContactInfo)
- **Sale** (SaleID, SaleDate, QuantityPurchased)
- Relationship **Purchases** connects Customer and Item through Sale.

Constraints:

Entity/Relationship	Constraint
Item	ItemID is unique (primary key)
Customer	CustomerID is unique (primary key)
Sale	SaleID unique; references CustomerID and ItemID
Purchases	Many-to-many relationship

This ER diagram effectively models the departmental store's inventory and sales record system, addressing the given requirements[assumption based on common database design principles].

(d) Design normalised tables in 3NF for the ER diagram drawn in part (c), with the required integrity constraints.

Solution: Normalized Tables in 3NF for Departmental Store ER Diagram

The ER diagram from part (c) has entities: **Item**, **Customer**, and the **Sale** relationship.

1. Item Table

Attribute	Description	Key/Constraint
ItemID	Unique identifier for each item	Primary Key (PK)
ItemName	Name of the item	Not Null
Price	Price of the item	Not Null
QuantityInStock	Current stock quantity	Not Null

2. Customer Table

Attribute	Description	Key/Constraint
CustomerID	Unique identifier for each customer	Primary Key (PK)
CustomerName	Name of the customer	Not Null
ContactInfo	Customer contact details	Nullable (optional)

3. Sale Table (Associative entity for purchases)

Attribute	Description	Key/Constraint
-----------	-------------	----------------

SaleID	Unique identifier for each sale	Primary Key (PK)
CustomerID	Identifier for customer who buys	Foreign Key (FK) referencing Customer(CustomerID)
ItemID	Identifier for item sold	Foreign Key (FK) referencing Item(ItemID)
SaleDate	Date of the sale	Not Null
QuantityPurchased	Number of items purchased	Not Null

Integrity Constraints

- **Primary Key Constraints:** Ensure unique identification of tuples in each table.
- **Foreign Key Constraints:**
 - CustomerID in Sale references Customer table.
 - ItemID in Sale references Item table.

These enforce referential integrity.
- **Not Null Constraints:** Ensure essential attributes always have values, like Price, QuantityInStock, and QuantityPurchased.
- **Data Validation:**
 - QuantityPurchased and QuantityInStock should always be non-negative.
 - SaleDate should be a valid date.

Normalization Justification

- Each table contains only attributes that depend on the key, the whole key, and nothing but the key, satisfying **3NF**.
- No transitive dependencies exist; for example, in Sale table, CustomerName is not present to avoid redundancy since it belongs in Customer table.

This design supports efficient storage, retrieval, and maintains data integrity aligned with the ER diagram specified in part (c).

(e) Explain the role of the primary index in a database system. Also, compare the primary indexes with the secondary indexes. What are clustering Indexes? Explain with the help of an example.

Solution: Role of Primary Index in a Database System

A **primary index** is an index on a set of fields that includes the primary key of a relation. It helps the database system **quickly locate records based on the primary key**, enhancing retrieval speed for queries that involve the primary key. The primary index is usually unique and sorted in the same order as the data file.

Comparison: Primary Index vs Secondary Index

Feature	Primary Index	Secondary Index
Based on	Primary key	Non-primary key attribute(s)
Uniqueness	Always unique	May be non-unique
Order	Data file is sorted based on primary index	Data file not necessarily sorted
Access Efficiency	Faster direct access using primary key	Used for accessing data via non-key fields
Usage	Main access path for records	Additional access paths

Clustering Index

A **clustering index** is an index on a non-key attribute that determines the **physical ordering of records in the data file**. Records with the same clustering key value are stored together (clustered) in the database.

Example

Consider a Students table with attributes:

- StudentID (Primary Key)
- Department

If the file is **physically sorted by the Department attribute**, and there is an index on Department, this index is called a clustering index since records with the same department appear together physically.

Summary

- Primary index improves access based on primary keys through sorted unique entries.
- Secondary index provides additional search paths using non-key attributes and may not correspond to physical order.
- Clustering index physically groups records with same attribute values to enhance retrieval by that attribute.

Question 2: (Covers Block 2)

(4+4+2+10=20 Marks)

(a) Consider a Relation: Course(CourseCode, CourseName, ProgrammeCode, ProgrammeName, CourseCredit, CourseDuration, ProgrammeDuration, ProgrammeCredit). Some of the constraints on the relation Course are:

- CourseCode uniquely identifies a course.
- ProgrammeCode is a unique code of a programme. A programme consists of many courses. A course can be part of multiple programmes.

- A Programme consists of compulsory courses and optional courses. To complete a Programme, a student must complete all the compulsory courses and optional courses, as per the total credit requirements of a Programme.

Perform the following tasks for the relation given above:

(i) What is the key to the relation?

Solution:

- The composite key is **(CourseCode, ProgrammeCode)** because:
 - CourseCode uniquely identifies a course but since a course can be part of multiple programmes,
 - ProgrammeCode identifies the programme.
So, together they uniquely identify a record.

(ii) Identify and list the functional dependencies in the relation.

Solution:

- CourseCode $\rightarrow$ CourseName, CourseCredit, CourseDuration
- ProgrammeCode $\rightarrow$ ProgrammeName, ProgrammeDuration, ProgrammeCredit
- (CourseCode, ProgrammeCode) $\rightarrow$ (Compulsory/Optional indicator) [if included]

(iii) Make an instance of this relation consisting of at least 8 to 10 records, showing possible redundancies.

Solution:

Course Code	CourseName	ProgrammeCode	ProgrammeName	Course Credit	CourseDuration	ProgrammeDuration	ProgrammeCredit
C01	Math	P01	BSc Mathematics	4	3 months	3 years	120
C02	Physics	P01	BSc Mathematics	3	3 months	3 years	120
C01	Math	P02	BSc Physics	4	3 months	3 years	130
C03	Chemistry	P02	BSc Physics	3	3 months	3 years	130

C04	Computer Sci	P03	BSc Comput er Sci	4	4 months	3 years	125
C01	Math	P03	BSc Comput er Sci	4	3 months	3 years	125
C05	English	P01	BSc Mathem atics	2	2 months	3 years	120
C02	Physics	P03	BSc Comput er Sci	3	3 months	3 years	125

Redundancy example: ProgrammeName, ProgrammeDuration, and ProgrammeCredit repeat for each course of the same programme.

(iv) **Decompose the relation Course into 2NF and 3NF relations.**

Solution: Decomposition steps:

- Identify partial dependencies and remove them (to achieve 2NF).
- Remove transitive dependencies (to achieve 3NF).

Tables in 2NF:

- **Course** (CourseCode, CourseName, CourseCredit, CourseDuration)
- **Programme** (ProgrammeCode, ProgrammeName, ProgrammeDuration, ProgrammeCredit)
- **CourseProgram** (CourseCode, ProgrammeCode, CompulsoryOptional)

Tables in 3NF:

Since fields in Course and Programme describe only respective entities, these are already in 3NF.

(b) **What is multi-valued dependency? Explain with the help of an example. How can it be used to decompose a relation into the 4th Normal Form? Explain with the help of an example. Also, explain the concept of the Join dependency with the help of an example.**

Solution:

1. What is Multi-Valued Dependency (MVD)

A **multi-valued dependency** occurs in a relation when, for a single value of attribute A, there are multiple independent values of attributes B and C. In simple terms,

attribute B and C are independently related to A, and their values do not depend on each other.

- Notation: $A \twoheadrightarrow B$ meaning "A multi-determines B"
- It signifies that for each value of A, there is a set of values for B and a set of values for C independently.

Example of MVD:

Consider the relation StudentCoursesLanguages(StudentID, Course, Language):

StudentID	Course	Language
S1	Math	English
S1	Math	French
S1	Science	English
S1	Science	French

- Here, StudentID multi-determines Course and Language independently; i.e., for Student S1, the courses and languages are independent sets, causing multi-valued dependency.

2. Decomposing into 4th Normal Form (4NF)

- **4NF** deals specifically with removing multi-valued dependencies.
- To decompose into 4NF, split the relation into two relations such that each contains one of the multi-valued attributes with the determinant.

Decomposition Example:

Split StudentCoursesLanguages into:

- StudentCourses(StudentID, Course)
- StudentLanguages(StudentID, Language)

This removes the MVD and avoids redundancy.

3. Join Dependency (JD)

- A **join dependency** is a generalization of multi-valued dependency.
- It means a relation can be reconstructed by joining multiple projections (sub-relations) of itself.
- Formally, a relation R will satisfy a join dependency if it equals the join of its projections on several subsets.

Example:

Consider relation $R(A, B, C)$ with projections $R_1(A, B)$, $R_2(B, C)$, and $R_3(A, C)$. If $R = R_1 \bowtie R_2 \bowtie R_3$, then R satisfies a join dependency.

- Join dependency helps define **5th Normal Form (5NF)** which removes join dependencies that are not implied by candidate keys.

(c) Explain the following terms with the help of an example of each: Views, Embedded SQL, Triggers and Dynamic SQL.

Solution: Explanation of Terms with Examples

1. Views

A **view** is a virtual table derived from one or more tables in the database. It does not store data itself but shows data dynamically based on a query.

Example:

```
CREATE VIEW StudentView AS
SELECT StudentID, StudentName FROM Students WHERE Department = 'CS';
```

This view lists all students from the Computer Science department.

2. Embedded SQL

Embedded SQL involves incorporating SQL statements within a host programming language such as C or Java. It allows database operations within the application code.

Example (in C):

```
EXEC SQL SELECT name INTO :studentName FROM Students WHERE id =
:studentID;
```

This fetches the student name into a C variable.

3. Triggers

A **trigger** is a procedure that automatically executes in response to certain events on a table, such as INSERT, UPDATE, or DELETE.

Example:

```
CREATE TRIGGER UpdateStock AFTER INSERT ON Sales
FOR EACH ROW
BEGIN
    UPDATE Inventory SET Quantity = Quantity - NEW.QuantitySold WHERE
    ItemID = NEW.ItemID;
END;
```

This trigger updates stock after a sale.

4. Dynamic SQL

Dynamic SQL refers to SQL statements constructed and executed at runtime, allowing flexible query generation based on conditions.

Example (in PL/SQL):

```
EXECUTE IMMEDIATE 'SELECT * FROM ' || table_name || ' WHERE id = :id'  
USING input_id;
```

This builds and runs an SQL query dynamically.

(d) Consider the following relational database:

Member (memberId, memberName, memberAddress, memberPhone)

BookIssued (memberID, BookID, IssueDate, returnDate)

Book (BookId, BookTitle, FirstAuthor)

The underlined attribute(s) in the relations given above form the primary key. In relation Member the attribute memberId is the unique identifier of a member. The purposes of other attributes in the Member relation are self-explanatory. You may define the domain of different attributes. Please note that the BookIssued relation has two foreign keys, namely memberID and BookID, which reference the Member and Book relations, respectively. Write and run the following SQL queries on the database:

(i) Create the tables with the primary and foreign key constraints.

Solution:

```
CREATE TABLE Member (  
    memberId VARCHAR(10) PRIMARY KEY,  
    memberName VARCHAR(100),  
    memberAddress VARCHAR(255),  
    memberPhone VARCHAR(15)
```

```
);
```

```
CREATE TABLE Book (  
    BookID VARCHAR(10) PRIMARY KEY,  
    BookTitle VARCHAR(255),  
    FirstAuthor VARCHAR(100)
```

```
);
```

```
CREATE TABLE BookIssued (  
    memberID VARCHAR(10),  
    BookID VARCHAR(10),  
    IssueDate DATE,
```

```
returnDate DATE,
PRIMARY KEY (memberID, BookID, IssueDate),
FOREIGN KEY (memberID) REFERENCES Member(memberId),
FOREIGN KEY (BookID) REFERENCES Book(BookID)
);
```

- (ii) **Insert at least 5 records each in the Member and Book tables and about 10 records in the BookIssued table.**

Solution:

```
-- Insert Members
INSERT INTO Member VALUES ('0001', 'Alice', '123 Main St', '555-1234');
INSERT INTO Member VALUES ('0002', 'Bob', '456 Park Ave', '555-5678');
INSERT INTO Member VALUES ('0003', 'Carol', '789 Elm St', '555-1234'); -- same
phone as Alice
INSERT INTO Member VALUES ('0004', 'David', '321 Oak St', '555-9012');
INSERT INTO Member VALUES ('0005', 'Eve', '654 Pine St', '555-3456');

-- Insert Books
INSERT INTO Book VALUES ('B001', 'Database Systems', 'XYZ');
INSERT INTO Book VALUES ('B002', 'Operating Systems', 'ABC');
INSERT INTO Book VALUES ('B003', 'Advanced Database Concepts', 'XYZ');
INSERT INTO Book VALUES ('B004', 'Networking Basics', 'DEF');
INSERT INTO Book VALUES ('B005', 'Database Design', 'PQR');

-- Insert BookIssued Records
INSERT INTO BookIssued VALUES ('0001', 'B001', '2023-07-01', '2023-07-10');
INSERT INTO BookIssued VALUES ('0001', 'B003', '2023-07-05', NULL);
INSERT INTO BookIssued VALUES ('0002', 'B002', '2023-07-10', '2023-07-20');
INSERT INTO BookIssued VALUES ('0003', 'B001', '2023-07-12', NULL);
INSERT INTO BookIssued VALUES ('0004', 'B004', '2023-07-15', '2023-07-22');
INSERT INTO BookIssued VALUES ('0005', 'B005', '2023-07-18', NULL);
INSERT INTO BookIssued VALUES ('0003', 'B005', '2023-07-20', NULL);
INSERT INTO BookIssued VALUES ('0002', 'B003', '2023-07-25', NULL);
INSERT INTO BookIssued VALUES ('0001', 'B002', '2023-07-30', NULL);
INSERT INTO BookIssued VALUES ('0005', 'B001', '2023-08-01', NULL);
```

- (iii) **List the name and phone number of all the members in lexicographical order.**

Solution:


```
SELECT memberName, memberPhone FROM Member ORDER BY  
memberName;
```

(iv) Find all the Books issued to the member whose memberID is "0001".

Solution:

```
SELECT B.BookTitle  
FROM BookIssued BI  
JOIN Book B ON BI.BookID = B.BookID  
WHERE BI.memberID = '0001';
```

(v) Find the list of those members who have not returned more than one book. (Assume that a book is not returned if returnDate is NULL)

Solution:

```
SELECT memberID  
FROM BookIssued  
WHERE returnDate IS NULL  
GROUP BY memberID  
HAVING COUNT(*) > 1;
```

(vi) Find the number of books whose first author's name is "XYZ".

Solution:

```
SELECT COUNT(*) AS NumBooks  
FROM Book  
WHERE FirstAuthor = 'XYZ';
```

(vii) Find the pair of members who have the same memberPhone.

Solution:

```
SELECT m1.memberName AS Member1, m2.memberName AS Member2,  
m1.memberPhone  
FROM Member m1  
JOIN Member m2 ON m1.memberPhone = m2.memberPhone AND m1.memberId <  
m2.memberId;
```

(viii) Find the list of Books whose title includes the term "Database".

Solution:

```
SELECT BookTitle  
FROM Book  
WHERE BookTitle LIKE '%Database%';
```

(ix) Find the title of the book that has been issued the most.

Solution:

```
SELECT B.BookTitle  
FROM BookIssued BI  
JOIN Book B ON BI.BookID = B.BookID  
GROUP BY BI.BookID, B.BookTitle  
ORDER BY COUNT(*) DESC  
LIMIT 1;
```

(x) Find the list of members who have not issued any books.

Solution:

```
SELECT memberName  
FROM Member M  
LEFT JOIN BookIssued BI ON M.memberId = BI.memberID  
WHERE BI.memberID IS NULL;
```

Question 3:

**IGNOU All Courses Solved Assignment PDFs/ Handwritten Scanned PDFs ,
Solved PYQs, Guess Papers, Help & Guide Books Available.**

WhatsApp: 92882 92884

(a) What are the ACID (Atomicity, Consistency, Isolation, Durability) properties of a transaction? Explain each property with a suitable example related to an online banking fund transfer. (4 Marks)

Solution: The ACID properties ensure reliable processing of database transactions. Each property with an example for online banking fund transfer:

1. Atomicity

- **Definition:** A transaction is “all or nothing.” Either all steps complete, or none are reflected in the database.
- **Example:** If you transfer ₹5,000 from Account A to Account B, the transaction should *debit* ₹5,000 from A and *credit* ₹5,000 to B. If the system fails after debiting but before crediting, the whole transaction is rolled back—so no money is deducted or credited.

2. Consistency

- **Definition:** Every transaction takes the database from one valid (consistent) state to another, following rules like constraints, triggers, and business logic.
- **Example:** After transferring ₹5,000, the total balance in both accounts remains correct (no money is “created” or “lost”), and account balances never go negative if negative balances are not allowed.

3. Isolation

- **Definition:** The effect of each transaction is invisible to others until it is complete, preventing interference even when multiple transactions run simultaneously.
- **Example:** While you are transferring ₹5,000 from A to B, another transaction cannot see an in-between state (where A is debited but B is not yet credited). Both transactions appear to run in strict sequence.

4. Durability

- **Definition:** Once a transaction completes successfully, its changes are permanent, even if there is a power failure or crash immediately after.
- **Example:** After a successful transfer, your updated account balances remain correct—even if the banking system crashes immediately after, you won't lose or gain money due to unsaved changes.

These properties collectively guarantee that even under failures or concurrency, every online banking transaction is safe, consistent, and reliable.

(b) Consider the following schedule involving two transactions, T1 and T2, where A and B are data items with initial values $A=1000$, $B=2000$.

Time	T1	T2
t1	READ(A)	
t2	$A = A - 100$	
t3		READ(A)
t4		$A = A * 1.1$
t5		WRITE(A)
t6		READ(B)
t7	WRITE(A)	
t8	READ(B)	
t9	$B = B + 100$	
t10	WRITE(B)	

(i) What is the final value of A and B after this schedule? (4 Marks)

Solution: Let's execute the schedule step by step, considering the initial values:

$A = 1000$, $B = 2000$

Step-wise Execution:

Time	T1 Action	T2 Action	State of A	State of B	Explanation
t1	READ(A)	-	1000	2000	T1 reads A
t2	$A = A - 100$ (T1: 900)	-	900*	2000	T1 calculates $A-100$
t3	-	READ(A)	900*	2000	T2 reads old $A=1000$
t4	-	$A = A * 1.1$ (T2: 1100)	900*	2000	T2 computes $A=1000*1.1$
t5	WRITE(A) (A updated to 900)	-	900	2000	T1 writes its version (900)
t6	-	READ(B)	900	2000	T2 reads B
t7	WRITE(A)	-	900	2000	T2 writes $A=1100$, but this is after T1's update
t8	-	READ(B)	900	2000	T1 reads B
t9	-	$B = B + 100$ (T1: 2100)	900	2100*	T1 computes $B+100$

t10	-	WRITE(B)	900	2100	T1 writes B
-----	---	----------	-----	------	-------------

Step-by-Step Execution:

- T1 reads $A = 1000$.
- T1 calculates $A = A - 100 = 900$.
- T2 reads **old** $A = 1000$.
- T2 calculates $A = A * 1.1 = 1100$ (based on the old value of A).
- T1 writes $A = 900$ to the database.
- T2 writes $A = 1100$ to the database – this overwrites T1's update.
- T1 and T2 read and operate on B, but only T1 updates B as $B = B + 100 = 2100$.
- T1 writes $B = 2100$.

Conclusion:

- The **final value of A** is **1100** (because T2's update was the last write to A).
- The **final value of B** is **2100** (updated by T1).

Explanation:

- T2's calculation of A used the old value of A (1000) before T1 wrote the updated value. Hence, T2 overwrites T1's earlier update to A.
- For B, only T1 modifies and writes the new value, so it becomes 2100 in the end.

Final answer:

- $A = 1100$
- $B = 2100$

(ii) Is this schedule serializable? Justify your answer. Identify the specific concurrency problem (e.g., Lost Update, Dirty Read) that occurs here. (4 Marks)

Solution:

1. Is the Schedule Serializable?

No, this schedule is not conflict serializable.

- When analyzing the schedule, we observe that both transactions T1 and T2 operate on the same data item A, with conflicting operations.
- T1 performs WRITE(A) with a value of 900 at time t5.
- T2, having read the old value of A (1000) before T1's write, later writes $A = 1100$ at t7, overwriting T1's update.
- This leads to a **conflict** because T2's write depends on outdated data that T1 has already modified.
- If we construct a **precedence (serialization) graph**, we would find a cycle:

- $T1 \rightarrow T2$ (because $T1$ writes A before $T2$ writes A , but $T2$'s operation depends on A before $T1$'s update)
- $T2 \rightarrow T1$ in some scenarios depending on interleaving (but here the overlapping writes cause inconsistency)
- The presence of such cycles means the schedule cannot be transformed into any serial schedule by swapping non-conflicting operations.

Therefore, the schedule is **not conflict serializable**; its execution is inconsistent with any serial order of transactions.

2. Specific Concurrency Problem Occurring

The specific concurrency problem occurring here is the **Lost Update Problem**.

Explanation:

- **Lost Update:** When two transactions simultaneously read and update the same data item, the first update may be overwritten by the second without being reflected in the final database.
- In this case, $T1$ updates A to 900, but $T2$, using the old value (1000), calculates and writes A as 1100, effectively losing $T1$'s update of 900.
- Thus, the final value of A does not reflect the update by $T1$ correctly.

(c) Explain the Two-Phase Locking (2PL) protocol. How does it ensure serializability?

Does 2PL prevent deadlocks? Explain with an example of how a deadlock can occur even when 2PL is used. (8 Marks)

Solution:

1. Two-Phase Locking (2PL) Protocol: Explanation

The **Two-Phase Locking (2PL) protocol** is a commonly used concurrency control protocol in database management systems to ensure that transactions are executed in a way that avoids conflicts and guarantees serializability (i.e., the outcome is the same as if transactions had executed one after another).

How 2PL Works:

- Each transaction must obtain the required locks (shared for reading, exclusive for writing) on data items before accessing them.
- The **2PL protocol divides the execution of a transaction into two distinct phases:**
 1. **Growing Phase:**
 - The transaction may obtain (acquire) locks but may **not release** any locks.

2. Shrinking Phase:

- The transaction may **release** locks but may **not acquire** any new locks.
- Once a transaction begins releasing any lock, it cannot acquire any more new locks.

2. How 2PL Ensures Serializability

Serializability means the schedule of transaction executions is equivalent in effect to some *serial* execution (one after the other).

- **2PL guarantees conflict-serializability:**
By enforcing the rule that no transaction can obtain a lock after it has released one, 2PL prevents certain problematic interleavings.
- **Why does this work?**
All lock and unlock operations occur in such a way that the serialization (precedence) graph underlying the transaction schedule is always acyclic. That is, transactions obtain all locks before releasing any, so conflicts that would introduce cycles—and hence non-serializable schedules—are impossible.

3. Does 2PL Prevent Deadlocks?

No, 2PL does NOT prevent deadlocks.

While 2PL enforces serializability, it *can* lead to situations where two or more transactions are each waiting for the other to release a lock, resulting in a deadlock where none can proceed.

4. Example: Deadlock under 2PL

Scenario:

Consider two transactions, **T1** and **T2**, with two data items **X** and **Y**.

- **Transaction T1:**
 1. lock(X) (acquires lock on X)
 2. lock(Y) (waits, because T2 has locked Y)
 3. ... (waiting)
- **Transaction T2:**
 1. lock(Y) (acquires lock on Y)
 2. lock(X) (waits, because T1 has locked X)
 3. ... (waiting)

Deadlock Illustration:

Step	T1 Action	T2 Action
1	lock(X) (granted)	

2		lock(Y) (granted)
3	lock(Y) (blocked)	
4		lock(X) (blocked)

Both transactions are in their **growing phase** (acquiring locks), but each is now waiting for a lock held by the other. Since neither will release its lock until it finishes, **both transactions are stuck in a deadlock**.

2PL is still maintained, but progress cannot occur without external intervention (e.g., a deadlock detector that aborts one transaction).

In conclusion:

- **Two-Phase Locking (2PL)** ensures serializability by strictly separating lock acquisition and release phases.
- **2PL does not prevent deadlocks**; simultaneous locking of resources by multiple transactions can still cause cyclic waiting. A deadlock occurs when, under 2PL, two or more transactions simultaneously wait for resources locked by each other.
- Deadlock detection and resolution (e.g., timeout or aborting transactions) must be handled separately by the DBMS.

Question 4:

Write short notes on any four of the following, explaining their purpose and key features. Provide an example where applicable. (4 x 5 = 20 Marks)

(i) Distributed Databases vs. Centralized Databases

Solution:

Purpose:

Both store and manage organizational data, but differ in where data resides and how it is managed.

Centralized Database:

- All data is stored at a single physical location (e.g., one server or data center).
- Users across the organization access data remotely over a network.
- **Features:**
 - Easier to manage, secure, and backup.
 - Risk: Single point of failure, potential bottleneck; can be less scalable.

Distributed Database:

- Data is physically spread across multiple locations (sites, networked computers, or servers).
- Each site may have its own DBMS but appears as a single database to users.
- **Features:**
 - Better scalability and availability.
 - Locally optimized access, improved reliability (if one site fails, others remain active).
 - More complex to manage integrity, consistency, and transaction control.

Example:

A global corporation stores customer data in regional data centers (distributed DB), while a small firm records all sales in one office server (centralized DB).

(ii) The Star Schema in Data Warehousing

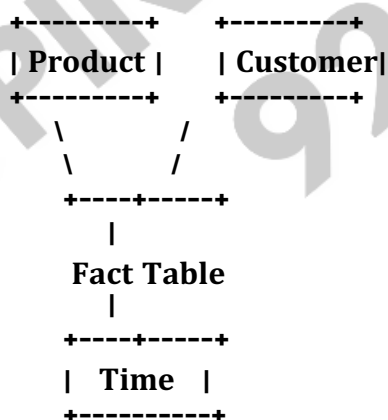
Solution: Purpose:

Used in data warehouses to organize data for efficient reporting and analytics.

Key Features:

- **Simple Design:**
 - Consists of a central **fact table** (e.g., Sales) for quantitative data.
 - Linked to multiple **dimension tables** (e.g., Time, Product, Customer) that describe key aspects.
- **Fact Table:**
 - Contains facts/measures, plus foreign keys to dimension tables.
- **Dimension Tables:**
 - Store descriptive (textual) information for context—usually denormalized for faster queries.

Example:



A sales data warehouse might track sales (fact) by time, product, and customer (dimensions).

(iii) NoSQL Databases (Explain one type, e.g., Document, Key-Value, or Graph)

Solution:

Purpose:

Designed for high scalability, flexibility, and handling large amounts of unstructured/semistructured data—unlike traditional relational databases.

Key Features:

- Do not use fixed schemas (schema-less or dynamic schemas).
- Allow storing data in various formats: key-value, document, column-family, or graph.
- Support efficient horizontal scaling for big data and real-time web applications.

Document Databases (e.g., MongoDB):

- Store data as documents (usually JSON or BSON).
- Each document can have different fields.
- Easy to represent hierarchical or variable data structures.
- Powerful querying and indexing capabilities.

(iv) Query Optimization in RDBMS

Solution:

Purpose:

Improves the performance of SQL queries by selecting the most efficient way to execute them.

Key Features:

- RDBMS parses and analyzes incoming SQL, generating multiple possible execution plans.
- The **query optimizer** estimates the cost of each plan (based on indexes, join methods, statistics) and chooses the optimal one.
- May use techniques like index selection, join reordering, and predicate pushdown.
- End goal: Fast retrieval and minimal resource use.

Example:

An SQL statement **SELECT * FROM Orders WHERE CustomerId = 10** could be executed:

- By scanning the entire Orders table (slow),
- Or by using an index on CustomerId (fast-optimizer prefers this).

(v) Log-Based Recovery and the role of Checkpoints

Solution:

Purpose:

Ensures database reliability and recoverability after failures (e.g., crashes, power loss).

Key Features:

- **Log-based recovery:**
 - Database maintains a **log file** recording all transaction updates (before and after values).
 - If a failure occurs, the system can **redo** (apply changes) or **undo** (roll back) transactions using the log.
- **Checkpoints:**
 - A snapshot of the current state periodically written to disk.
 - Marks a point from which recovery can start, making the process much faster by not needing to scan the entire log from the beginning.

Example:

If a server crashes after a transfer transaction, on restart:

- The DBMS consults the log and the latest checkpoint,
- Recovers/rolls back incomplete transactions to ensure consistency,
- Restores the system to a stable state without data loss.